

IFSC – Instituto Federal de Santa Catarina

Campus São José

Curso de Engenharia de Telecomunicações

Sistemas Operacionais

Gabarito – Exercícios: Gestão de Tarefas

Baseado no livro *“Sistemas Operacionais: Conceitos e Mecanismos”*
Prof. Carlos A. Maziero (UFPR)

Professor: Rogério Pereira Junior
rogerio.pereira@ifsc.edu.br

07/04/2026

Questão 1 – Programa *vs.* Tarefa (Processo)

Programa é uma entidade *estática*: trata-se de um conjunto de instruções armazenadas em disco (um arquivo executável). Ele não possui estado de execução, registradores ou contexto próprio.

Tarefa (processo) é uma entidade *dinâmica*: é a instância de um programa *em execução*. Possui estado (contador de programa, pilha, espaço de endereçamento, recursos abertos, etc.) e avança ao longo do tempo conforme o processador executa suas instruções.

Analogia: Pense em uma **receita de bolo** (programa) e no **ato de fazer o bolo** (processo/tarefa). A receita é estática – ela existe no caderno independentemente de alguém a seguir. O processo de fazer o bolo é dinâmico: envolve um cozinheiro (processador), ingredientes (dados na memória), utensílios (recursos do sistema) e progride passo a passo. A mesma receita pode ser executada simultaneamente por dois cozinheiros diferentes, gerando dois processos distintos a partir de um único programa.

Questão 2 – Cinco estados de uma tarefa

Os cinco estados principais de uma tarefa em um sistema de tempo compartilhado são:

1. **Nova (N):** A tarefa acaba de ser criada. O SO está alocando e inicializando as estruturas internas (TCB, espaço de endereçamento). A tarefa ainda não está apta a competir pelo processador.
2. **Pronta (P):** A tarefa está completamente inicializada e pronta para ser executada. Ela aguarda na fila de prontas até que o escalonador lhe conceda o processador.
3. **Executando (E):** O processador está efetivamente executando as instruções da tarefa. Em um sistema com n processadores, podem existir até n tarefas neste estado simultaneamente.
4. **Suspensa (S):** A tarefa está aguardando algum evento externo para poder continuar (conclusão de E/S, liberação de semáforo, temporizador, etc.). Ela não compete pelo processador enquanto está suspensa.
5. **Terminada (T):** A tarefa concluiu sua execução. O SO libera os recursos alocados (memória, descritores de arquivo, etc.) e remove a entrada da tabela de processos.

Transição Executando → Pronta: Ocorre quando a tarefa *esgota seu quantum* de tempo (preempção por tempo). O escalonador retira o processador da tarefa atual e a devolve à fila de prontas, concedendo o processador a outra tarefa. A tarefa não

bloqueou – ela simplesmente perdeu a vez. Também pode ocorrer por *preempção por prioridade*, quando uma tarefa de maior prioridade torna-se pronta.

Questão 3 – *Time Sharing*

Time sharing (tempo compartilhado) é uma técnica em que o SO divide o tempo de CPU em pequenos intervalos chamados **quanta** e os distribui ciclicamente entre as tarefas prontas. Cada tarefa recebe um quantum para executar; ao final, o processador é preemptado e concedido à próxima tarefa.

Importância:

- Permite que *múltiplos usuários ou processos* utilizem o computador de forma aparentemente simultânea (concorrência).
- Garante *responsividade*: nenhuma tarefa monopoliza o processador indefinidamente.
- Possibilita a execução de sistemas *interativos*, nos quais o usuário espera respostas rápidas.
- É a base dos sistemas operacionais modernos de uso geral (Linux, Windows, macOS).

Questão 4 – Conceito de Processo

Um **processo** é a abstração central de um sistema operacional multiprogramado. Formalmente, pode ser definido como:

“Uma instância de um programa em execução, incluindo o estado atual da execução (registradores, contador de programa, pilha) e todos os recursos associados a ela (memória, arquivos abertos, conexões de rede, etc.).”

Do ponto de vista do SO, um processo é a **unidade básica de alocação de recursos**: toda memória, tempo de CPU e dispositivos de E/S são atribuídos a processos. Cada processo possui um identificador único (*PID – Process Identifier*) e é representado internamente por seu *PCB/TCB*.

Questão 5 – Processo como “contêiner de recursos” e *threads*

Por que “contêiner de recursos”?

Historicamente, processo era sinônimo de *unidade de execução* (havia apenas um fluxo de controle por processo). Com a introdução de *threads*, o processo passou a abrigar *múltiplos fluxos de execução* (threads) que compartilham o mesmo espaço de endereçamento e recursos. Assim, o processo tornou-se o **contêiner** que agrupa e

isola recursos, enquanto as threads são as unidades efetivas de execução dentro dele.

O que as threads de um mesmo processo compartilham:

- Espaço de endereçamento (código, dados globais e heap)
- Descritores de arquivos abertos
- Variáveis globais e estáticas
- Conexões de rede e outros recursos de E/S
- Permissões e credenciais do processo

O que cada thread possui individualmente:

- Contador de programa (PC)
- Pilha de execução própria
- Conjunto de registradores
- Estado de execução (pronta, executando, suspensa)

Questão 6 – Transições de estado

Transição	Possível?	Exemplo / Justificativa
$E \rightarrow P$	Sim	A tarefa esgota seu quantum (preempção por tempo) ou é preemptada por uma tarefa de maior prioridade.
$E \rightarrow S$	Sim	A tarefa solicita uma operação de E/S (leitura de disco, espera por rede) ou tenta adquirir um semáforo já em uso.
$S \rightarrow E$	Não	Uma tarefa suspensa não pode ir diretamente para Executando. Ela precisa primeiro voltar ao estado Pronta e aguardar o escalonador.
$P \rightarrow N$	Não	O ciclo de vida é unidirecional nesse sentido; uma tarefa pronta não pode “regredir” ao estado Nova.
$S \rightarrow T$	Não	Em condições normais, uma tarefa suspensa não termina diretamente. Ela retorna ao estado Pronta e depois Executando antes de terminar. (Exceção: sinal de kill externo pode forçar terminação, mas isso é uma transição forçada pelo SO, não pelo ciclo normal.)
$E \rightarrow T$	Sim	A tarefa executa a chamada de sistema <code>exit()</code> ou retorna da função <code>main()</code> , encerrando voluntariamente.
$N \rightarrow S$	Não	Uma tarefa recém-criada vai para o estado Pronta após a inicialização; não pode ir diretamente para Suspensa.
$P \rightarrow S$	Não	Para bloquear (ir a Suspensa), a tarefa precisa estar Executando e fazer uma chamada bloqueante. Uma tarefa Pronta não executa código e, portanto, não pode bloquear por conta própria.

Questão 7 – Associação de afirmações aos estados

Estado	Afirmação
[N] Nova	O código da tarefa está sendo carregado.
[P] Pronta	As tarefas são ordenadas por prioridades.
[E] Executando	A tarefa sai deste estado ao solicitar uma operação de entrada/saída.
[T] Terminada	Os recursos usados pela tarefa são devolvidos ao sistema.
[P] Pronta	A tarefa vai a este estado ao terminar seu quantum.
[P] Pronta	A tarefa só precisa do processador para poder executar.
[S] Suspensa	O acesso a um semáforo em uso pode levar a tarefa a este estado.
[E] Executando	A tarefa pode criar novas tarefas.
[E] Executando	Há uma tarefa neste estado para cada processador do sistema.
[S] Suspensa	A tarefa aguarda a ocorrência de um evento externo.

Questão 8 – Múltiplas *threads* (múltipla escolha)

Alternativa correta: (b)

Threads de um mesmo processo compartilham arquivos e memória (espaço de endereçamento, heap, dados globais, descritores de arquivos).

Análise das demais:

- (a) **Errada.** Cada thread *não* possui seu próprio espaço de endereçamento; todas as threads de um mesmo processo compartilham o mesmo espaço.
- (c) **Errada.** Uma thread não corresponde necessariamente a um processo; várias threads podem existir dentro de um único processo.
- (d) **Errada.** Cada thread possui seu próprio contador de programa (PC) e conjunto de registradores.

Questão 9 – Modelos de implementação de *threads* (múltipla escolha)

Alternativa correta: (b) – Apenas I e II

- **I. Verdadeira.** No modelo N:1 (*many-to-one*), todas as threads de usuário são multiplexadas em um único thread de núcleo. Uma chamada de sistema bloqueante bloqueia o thread de núcleo, suspendendo todas as demais threads

do processo.

- **II. Verdadeira.** No modelo 1:1 (*one-to-one*), cada thread de usuário possui um thread de núcleo correspondente. O escalonador do SO pode escalona-los independentemente, inclusive em múltiplos processadores.
- **III. Falsa.** O modelo N:M (*many-to-many*) não elimina a necessidade de threads de núcleo; ao contrário, ele multiplexa N threads de usuário em M threads de núcleo ($M \leq N$). Threads de núcleo continuam sendo necessários; o modelo apenas os usa de forma mais eficiente.

Questão 10 – TCB (*Task Control Block*)

O que é: O TCB (*Task Control Block*), também chamado de PCB (*Process Control Block*), é a estrutura de dados mantida pelo sistema operacional para representar e gerenciar uma tarefa/processo.

Para que serve: É o “cartão de identidade” da tarefa no SO. Permite que o escalonador salve e restaure o contexto de execução (troca de contexto), que o SO controle os recursos alocados e que múltiplas tarefas coexistam de forma segura e organizada.

O que contém (principais campos):

- **Identificação:** PID, PID do pai, UID/GID do usuário proprietário.
- **Estado:** estado atual da tarefa (Nova, Pronta, Executando, Suspensa, Terminada).
- **Contexto de hardware:** valores dos registradores (PC, SP, registradores gerais, flags) – salvos na troca de contexto.
- **Informações de escalonamento:** prioridade, tempo de CPU consumido, quantum restante, política de escalonamento.
- **Gerência de memória:** ponteiro para a tabela de páginas ou mapa de segmentos.
- **Recursos de E/S:** tabela de descritores de arquivos abertos, conexões de rede.
- **Informações de contabilidade:** tempo de CPU, tempo real decorrido, limites de recursos.
- **Ponteiros de fila:** ponteiros para o próximo/anterior TCB nas filas do escalonador.

Questão 11 – Análise do código com `fork()`

Código analisado

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 int main() {
5     int x = 0;
6     fork();          // Fork 1
7     x++;
8     sleep(5);
9     wait(0);
10    fork();           // Fork 2
11    wait(0);
12    sleep(5);
13    x++;
14    printf("Valor de x: %d\n", x);
15    return 0;
16 }
```

Rastreamento da execução

Passo 1 – Fork 1 (linha 6): O processo original P0 chama `fork()`, criando P1. A partir daqui, P0 e P1 executam em paralelo com $x = 0$.

Passo 2 – `x++` (linha 7): Tanto P0 quanto P1 incrementam x : $x = 1$ em ambos.

Passo 3 – `sleep(5)` (linha 8): P0 e P1 dormem 5 segundos em paralelo (duração efetiva: ~5s).

Passo 4 – `wait(0)` (linha 9): P0 espera por filhos: P0 aguarda P1 terminar. P1 não tem filhos, então `wait` retorna imediatamente (ECHILD).

Passo 5 – Fork 2 (linha 10):

- **P1** (sem filhos) chama `fork()`, criando P2. ($x=1$ em ambos P1 e P2)
- **P0** só chega aqui quando P1 terminar (bloqueado no `wait`).

Passo 6 – `wait(0)` (linha 11) e `sleep(5)` (linha 12):

- P1 espera P2 terminar. P2 não tem filhos, `wait` retorna imediatamente, P2 dorme 5s, incrementa $x = 2$, imprime e termina.
- P1 é liberado do `wait`, dorme 5s, incrementa $x = 2$, imprime e termina.
- Então P0 é liberado do seu `wait` (linha 9), chama `fork` criando P3. P0 e P3 seguem o mesmo fluxo (`wait` retorna imediato para P3, `sleep 5s`, `x++`, `print`).

Saída na tela

São impressas **4 linhas**:

Processo	Saída
P2	Valor de x: 2
P1	Valor de x: 2
P3	Valor de x: 2
P0	Valor de x: 2

Todos imprimem $x = 2$ pois cada processo incrementa x duas vezes (linhas 7 e 13).

Duração aproximada

- **Fase 1:** `sleep(5)` de P0 e P1 em paralelo $\approx 5s$.
- **Fase 2:** P1 aguarda P2 terminar. P2 faz `sleep(5)` $\approx 5s$. P1 então dorme 5s (paralelo com P2? Não – P1 está no `wait`). Após P2 terminar, P1 dorme 5s $\approx 5s$ adicionais. Total do ramo P1/P2: **10s** (contados após a fase 1).
- **Fase 3:** P0 é liberado do `wait` após P1 terminar (10s depois da fase 1). P0 cria P3. P3 não tem filhos, `wait` imediato, ambos dormem 5s. Total fase 3: **5s**.

Duração total (sequencial do ponto de vista de P0):

$$T_{total} \approx 5 + 10 + 5 = 20 \text{ segundos}$$

Questão 12 – O que são *threads*?

Threads (ou *linhas de execução*) são fluxos de controle independentes que existem *dentro* de um processo. Enquanto um processo é a unidade de alocação de recursos, a thread é a unidade de execução (escalonamento).

Para que servem:

- Permitir que um único processo execute múltiplas atividades **concorrentemente** (ex.: interface gráfica respondendo ao usuário enquanto processa dados em background).
- Explorar **múltiplos núcleos** de processador para paralelismo real.
- Compartilhar dados entre fluxos de execução de forma eficiente (sem necessidade de IPC complexo).
- Reduzir o custo de criação/troca de contexto em relação a processos separados.

Cada thread possui seu próprio contador de programa, pilha e registradores, mas compartilha com as demais threads do mesmo processo o espaço de endereçamento, arquivos abertos e outros recursos.

Questão 13 – Vantagens e desvantagens de *threads* vs. processos

Vantagens das threads	Desvantagens das threads
Criação e destruição muito mais rápidas que processos (sem alocação de novo espaço de endereçamento).	Compartilham memória: um erro em uma thread pode corromper dados de outras.
Troca de contexto entre threads do mesmo processo é mais barata (não requer mudança de mapa de memória).	Programação concorrente é complexa: <i>race conditions</i> , <i>deadlocks</i> e problemas de sincronização.
Comunicação entre threads é simples (variáveis compartilhadas), ao contrário da IPC entre processos.	Uma thread com defeito pode derrubar o processo inteiro (sem isolamento de falhas).
Permitem explorar paralelismo em sistemas multiprocessadores.	No modelo N:1, uma thread bloqueante paralisa todas as demais.
Responsividade em aplicações interativas (ex.: UI thread separado de thread de processamento).	Depuração mais difícil: erros podem ser não-determinísticos (dependem do escalonamento).

Questão 14 – Escalonamento de tarefas

Dados das tarefas

	t1	t2	t3	t4	t5
Ingresso	0	0	3	5	7
Duração	5	4	5	6	4
Prioridade	2	3	5	9	6

Tempo total de CPU = $5+4+5+6+4 = 24$ unidades.

Para cada política calculamos:

- **Turnaround** (T_t) = tempo de término – tempo de ingresso
- **Waiting time** (T_w) = turnaround – duração

a) FCFS Cooperativa

Ordem de chegada: t1 (0), t2 (0) $\Rightarrow$ menor índice primeiro.

Sequência: t1(0–5), t2(5–9), t3(9–14), t4(14–20), t5(20–24)

Tarefa	Ingresso	Duração	Término	T_t	T_w
t1	0	5	5	5	0
t2	0	4	9	9	5
t3	3	5	14	11	6
t4	5	6	20	15	9
t5	7	4	24	17	13
Média				11,4	6,6

b) SJF Cooperativa

Escolhe a tarefa de *menor duração* entre as prontas quando o processador fica livre. Não há preempção.

- $t = 0$: prontas {t1(5), t2(4)} $\Rightarrow$ t2 executa (0–4)
- $t = 4$: prontas {t1(5), t3(5)} $\Rightarrow$ empate, menor índice: t1 (4–9)
- $t = 9$: prontas {t3(5), t4(6)} $\Rightarrow$ t3 (9–14)
- $t = 14$: prontas {t4(6), t5(4)} $\Rightarrow$ t5 (14–18)
- $t = 18$: t4 (18–24)

Tarefa	Ingresso	Duração	Término	T_t	T_w
t1	0	5	9	9	4
t2	0	4	4	4	0
t3	3	5	14	11	6
t4	5	6	24	19	13
t5	7	4	18	11	7
Média				10,8	6,0

c) SJF Preemptiva – SRTF (*Shortest Remaining Time First*)

Preempta a tarefa em execução quando chega uma nova tarefa com *tempo restante menor*.

Linha do tempo:

- $t = 0$: prontas {t1(5), t2(4)} $\Rightarrow$ t2 executa
- $t = 3$: chega t3(5); t2 restante=1, t3=5 $\Rightarrow$ t2 continua
- $t = 4$: t2 termina; prontas {t1(5), t3(5)} $\Rightarrow$ empate $\Rightarrow$ t1 executa
- $t = 5$: chega t4(6); t1 restante=4, t4=6 $\Rightarrow$ t1 continua
- $t = 7$: chega t5(4); t1 restante=2, t5=4 $\Rightarrow$ t1 continua
- $t = 9$: t1 termina; prontas {t3(5), t4(6), t5(4)} $\Rightarrow$ t5 executa

- $t = 13$: t5 termina; prontas $\{t3(5), t4(6)\} \Rightarrow t3$ executa
- $t = 18$: t3 termina; t4 executa (18–24)

Tarefa	Ingresso	Duração	Término	T_t	T_w
t1	0	5	9	9	4
t2	0	4	4	4	0
t3	3	5	18	15	10
t4	5	6	24	19	13
t5	7	4	13	6	2
Média				10,6	5,8

d) PRIO Cooperativa (maior valor = maior prioridade)

A tarefa com maior prioridade dentre as prontas é executada até terminar.

- $t = 0$: prontas $\{t1(p=2), t2(p=3)\} \Rightarrow t2$ executa (0–4)
- $t = 4$: prontas $\{t1(p=2), t3(p=5)\} \Rightarrow t3$ executa (4–9)
- $t = 9$: prontas $\{t1(p=2), t4(p=9), t5(p=6)\} \Rightarrow t4$ executa (9–15)
- $t = 15$: prontas $\{t1(p=2), t5(p=6)\} \Rightarrow t5$ executa (15–19)
- $t = 19$: t1 executa (19–24)

Tarefa	Ingresso	Duração	Término	T_t	T_w
t1	0	5	24	24	19
t2	0	4	4	4	0
t3	3	5	9	6	1
t4	5	6	15	10	4
t5	7	4	19	12	8
Média				11,2	6,4

e) PRIO Preemptiva

Preempta sempre que chega uma tarefa com prioridade maior que a em execução.

- $t = 0$: $\{t1(p=2), t2(p=3)\} \Rightarrow t2$ executa
- $t = 3$: chega $t3(p=5) > t2(p=3) \Rightarrow$ preempta t2; t3 executa
- $t = 5$: chega $t4(p=9) > t3(p=5) \Rightarrow$ preempta t3; t4 executa
- $t = 7$: chega $t5(p=6) < t4(p=9) \Rightarrow$ t4 continua
- $t = 11$: t4 termina; prontas $\{t1(2), t2(1 \text{ restante}), t3(3 \text{ restantes}), t5(4)\} \Rightarrow$ t5(p=6) executa

- $t = 15$: t5 termina; $\Rightarrow$ t3(p=5) continua com 3 restantes (15–18)
- $t = 18$: t3 termina; $\Rightarrow$ t2(p=3) continua com 1 restante (18–19)
- $t = 19$: t2 termina; $\Rightarrow$ t1(p=2) (19–24)

Tarefa	Ingresso	Duração	Término	T_t	T_w
t1	0	5	24	24	19
t2	0	4	19	19	15
t3	3	5	18	15	10
t4	5	6	11	6	0
t5	7	4	15	8	4
Média				14,4	9,6

f) Round-Robin com $tq = 2$, sem envelhecimento

Cada tarefa recebe no máximo 2 unidades de CPU por vez. Chegadas são inseridas no fim da fila.

Simulação (fila de prontas em ordem):

Intervalo	Tarefa	Restante após
0–2	t1	t1=3, t2=4
2–4	t2	t1=3, t2=2, t3=5
4–6	t1	t1=1, t2=2, t3=5
6–8	t2	t1=1, t3=5, t4=6
8–9	t1 (1 restante)	t3=5, t4=6
9–11	t3	t3=3, t4=6, t5=4
11–13	t4	t3=3, t4=4, t5=4
13–15	t5	t3=3, t4=4, t5=2
15–17	t3	t3=1, t4=4, t5=2
17–19	t4	t3=1, t4=2, t5=2
19–21	t5	t3=1, t4=2
21–22	t3 (1 restante)	t4=2
22–24	t4	t4=0

Tarefa	Ingresso	Duração	Término	T_t	T_w
t1	0	5	9	9	4
t2	0	4	8	8	4
t3	3	5	22	19	14
t4	5	6	24	19	13
t5	7	4	21	14	10
Média				13,8	9,0

Questão 15 – Modelos de threads (N:1, 1:1, N:M)

Modelo	Afirmações
(a) N:1	(a) Tem a implementação mais simples, leve e eficiente (d) Não permite explorar a presença de várias CPUs pelo mesmo processo (f) Geralmente implementado por bibliotecas (g) Se um thread bloquear, todos os demais têm de esperar por ele
(b) 1:1	(c) Pode impor uma carga muito pesada ao núcleo (h) Cada thread no nível do usuário tem sua correspondente dentro do núcleo
(c) N:M	(b) Multiplexa os threads de usuário em um pool de threads de núcleo (e) Permite uma maior concorrência sem impor muita carga ao núcleo (i) É o modelo com implementação mais complexa

Questão 16 – Escalonador e fila de prontas

O **escalonador** (*scheduler*) é o componente do SO responsável por decidir qual tarefa da fila de prontas deve receber o processador e por quanto tempo.

Papel:

- Selecionar a próxima tarefa a executar com base em uma **política de escalonamento**.
- Realizar a **troca de contexto**: salvar o estado (registradores, PC) da tarefa atual no seu TCB e carregar o estado da nova tarefa.
- Garantir critérios como **justiça** (todas as tarefas progridem), **eficiência** (CPU bem utilizada) e **responsividade** (tarefas interativas têm latências baixas).

Como decide: Depende da política adotada:

- **FCFS**: escolhe a tarefa mais antiga na fila.
- **SJF/SRTF**: escolhe a de menor tempo (restante) de execução.
- **Prioridade**: escolhe a de maior prioridade.
- **Round-Robin**: divide o tempo em quanta iguais, atendendo as tarefas em rodízio.

Questão 17 – Escalonamento Round-Robin

No **Round-Robin (RR)**, o escalonador atribui a cada tarefa pronta um intervalo de tempo fixo chamado **quantum** (tq). Ao final do quantum, a tarefa é preemptada e vai ao fim da fila de prontas, e a próxima tarefa recebe o processador.

Características:

- Preemptivo por tempo.
- Justo: todas as tarefas recebem frações iguais de CPU.
- Bom para sistemas interativos.
- Desempenho depende do tamanho do quantum: muito pequeno $\Rightarrow$ overhead de troca; muito grande $\Rightarrow$ comportamento similar ao FCFS.

Exemplo ($tq = 2$, tarefas A(4), B(3), C(2), todas chegando em $t=0$):

t:	0-2	2-4	4-6	6-8	8-9	9-11
	A	B	C	A	B	-

A termina em $t=8$, B em $t=9$, C em $t=6$.

Questão 18 – Técnica de *Aging* (Envelhecimento)

O que é: *Aging* (envelhecimento) é uma técnica usada em escalonadores baseados em prioridade para **evitar starvation** (inanição) de tarefas de baixa prioridade.

Para que serve: Em políticas por prioridade pura, tarefas de baixa prioridade podem nunca ser executadas se o sistema estiver sempre ocupado com tarefas de alta prioridade. O aging resolve esse problema.

Como funciona: A prioridade de uma tarefa *aumenta gradualmente* conforme ela permanece esperando na fila de prontas sem ser executada. Após um tempo suficiente, sua prioridade sobe o bastante para que ela seja selecionada pelo escalonador.

Exemplo: Uma tarefa com prioridade 2 que aguarda há 10 unidades de tempo pode ter sua prioridade elevada para 5 (aumentando 1 a cada 2 unidades de espera). Eventualmente, ela será atendida antes de novas tarefas de prioridade média.

A operação inversa (reduzir prioridade com o tempo de execução) é chamada de **decaimento** e é usada para favorecer tarefas que ainda não receberam CPU.

Desafio – Contagem de “X” impressos

Linha de comando

```
a.out 4 3 2 1  $\Rightarrow$  argc=5, argv = {"a.out", "4", "3", "2", "1"}
N = atoi(argv[argc-2]) = argv[3] = "2"  $\Rightarrow$  N = 2
```

Análise do código

Parte 1 – loop de fork:

```
1 for (i = 0; i < N; i++) {    // N = 2 iteracoes
2     pid[i] = fork();
3 }
```

A cada `fork()`, o número de processos dobra (pai e filho continuam o loop):

- Antes do loop: 1 processo (P0).
- Após `fork i=0`: 2 processos (P0, P1).
- Após `fork i=1`: 4 processos (P0, P1, P2, P3).

Após o loop, existem **4 processos**. Para cada processo, os valores de `pid[0]` e `pid[1]` são:

Processo	pid[0]	pid[1]
P0 (original)	PID_P1 $\neq$ 0	PID_P2 $\neq$ 0
P1 (filho fork0)	0	PID_P3 $\neq$ 0
P2 (filho fork1 de P0)	PID_P1 $\neq$ 0	0
P3 (filho fork1 de P1)	0	0

Parte 2 – fork extra:

```
1 if (pid[0] != 0 && pid[N-1] != 0) {    // pid[0] != 0 AND pid[1]
2     != 0
3     pid[N] = fork();    // pid[2] = fork()
4 }
5
```

Somente P0 satisfaz a condição (ambos $\neq$ 0). P0 chama `fork()`, criando P4.

Resultado: 5 processos no total: P0, P1, P2, P3, P4.

Parte 3 – `printf("X")`: Todos os 5 processos chegam ao `printf`.

Número de “X” impressos = 5